

Robot Remote Control Working Group

Moorthi PV

Request for Comments: RRCF-0001

Indo-Mars Technologies

Category: Standards Track

Version 0.4 DRAFT September 23, 2026

a

RRCF: Robot Remote Control Format

Operator Interface Declaration Standard for Physical AI

Version 1.0 Draft — Apache License 2.0

We are entering an era of robot proliferation without historical precedent. From sub-millimeter medical nanorobots to multi-ton industrial humanoids, from agricultural swarms covering thousands of acres to personal service robots in every home — the diversity and scale of robotic deployment will reach billions of units within this decade.

In every deployment context, a human in the loop is inevitable. A surgeon monitors a medical robot. A farmer supervises an agricultural swarm. A warehouse manager oversees a mixed fleet. A first responder needs to halt a malfunctioning robot. A child interacts with a home assistant. An AI system generates commands for a physical agent. At every level — from direct teleoperation to high-level supervision to emergency intervention — humans will interact with robots constantly, every day, at all times.

Today, every robot requires its own controller, its own interface, its own training. This fragmentation is unsustainable at scale. Just as the proliferation of automobiles required standardized controls so that any licensed driver could operate any vehicle safely, the proliferation of billions of robots demands a global common operator interface standard. Without it, robot operation remains a specialist skill, safety is inconsistent, fleet management is fragmented, and the potential of Physical AI is bottlenecked by interface incompatibility.

Billions of robots are coming — from nano to humanoid. Every human will deal with robots every day. A global common operator interface standard is not optional. RRCF is that standard.

Abstract

We present RRCF (Robot Remote Control Format), the first Operator Interface Declaration Standard for robotic platforms. RRCF defines a machine-readable file format (.rrcf) in which robot manufacturers declare their complete operator control interface — morphology category, locomotion modes, skill repertoire, HUD display fields, joystick axis semantics, input modalities (touch, HID gamepad, voice, gesture, XR), attachment control panels, custom operator controls, safety limits, and transport endpoints. Any RRCF-compliant controller — web UI, fleet dashboard, or VLA model — reads this declaration and renders the correct operator interface without per-robot code. Human operators use the rendered web UI; AI systems load the .rrcf profile directly at session start via retrieval-augmented generation (RAG for robots). RRCF complements URDF, VDA 5050, Open-RMF, and NVIDIA Halos as the missing operator interface layer. Published Apache 2.0.

1. Motivation — The Human-Robot Interface Imperative

We are entering an era of robot proliferation without historical precedent. From sub-millimeter medical nanorobots to multi-ton industrial humanoids, from agricultural swarms covering thousands of acres to personal service robots in every home — the diversity and scale of robotic deployment will reach billions of units within this decade.

In every deployment context, a human in the loop is inevitable. A surgeon monitors a medical robot. A farmer supervises an agricultural swarm. A warehouse manager oversees a mixed fleet. A first responder needs to halt a malfunctioning robot. A child interacts with a home assistant. An AI system generates commands for a physical agent. At every level — from direct teleoperation to high-level supervision to emergency intervention — humans will interact with robots constantly, every day, at all times.

Today, every robot requires its own controller, its own interface, its own training. This fragmentation is unsustainable at scale. Just as the proliferation of automobiles required standardized controls so that any licensed driver could operate any vehicle safely, the proliferation of billions of robots demands a global common operator interface standard. Without it, robot operation remains a specialist skill, safety is inconsistent, fleet management is fragmented, and the potential of Physical AI is bottlenecked by interface incompatibility.

Billions of robots are coming — from nano to humanoid. Every human will deal with robots every day. A global common operator interface standard is not optional. RRCF is that standard.

This extends to on-road autonomous and electric vehicles — cars, robotaxis, and EV shuttle fleets are, at the operator-interface layer, simply another class of RRCF-compliant vehicle. Following SAE J3016, RRCF treats remote assistance (advisory guidance — route confirmation, obstacle waypoint grants, permission-to-proceed — without direct actuation) and remote driving (direct real-time control of steering, throttle, and brake) as two distinct, correctly-scoped operator relationships, rather than folding both into the generic term “teleoperation.”

2. Core Definition

RRCF is the Operator Interface Declaration Standard that defines how any robot is controlled — by human operators through a unified web UI, or by AI systems (VLA) through direct .rrcf profile loading at runtime — and also includes fleet management, collection and integration, safety guardrails, and choreography standards, complementing URDF, VDA 5050, and other existing standards to support most parts of Physical AI.

Key distinction: RRCF is an Operator Interface Declaration. VDA 5050 and Open-RMF are fleet mission assignment protocols. They operate at different layers and are complementary. RRCF sits above fleet protocols as the human/AI-facing operator layer.

3. The Five Pillars

1. REMOTE CONTROL STANDARD — One unified UI for any robot, any morphology
2. FLEET MANAGEMENT STANDARD — Complements VDA 5050 / Open-RMF (not competes)
3. COLLECTION & INTEGRATION STANDARD — Universal wire format for any external system
4. SAFETY STANDARD — E-stop,

watchdog, geofence standardized across all robots5. CHOREOGRAPHY STANDARD — Synchronized multi-robot missions (.rrcm files)

3.1 Pillar 1 — Remote Control Standard (Primary Purpose)

RRCF's primary purpose is a unified operator interface for any robot. A robot manufacturer publishes a .rrcf file. Any RRCF-compliant web controller reads it and renders the correct interface — joysticks, mode buttons, skill actions, HUD strip, button legend, attachment tabs, custom controls — without per-robot code. Operator means human (web UI) or AI system (VLA via .rrcf profile).

This pattern extends directly to on-road vehicles: an RRCF-compliant EV or robotaxi declares a road_vehicle primary category with steering, throttle, and brake control bindings (Section 6, Section 7.3), and the same operator interface renders either a full remote-driving control panel or a remote-assistance advisory panel depending on the declared authority tier (Section 3.4, Section 11).

3.2 Pillar 2 — Fleet Management Standard

RRCF complements VDA 5050 and Open-RMF — it does not compete with them. VDA 5050 defines how fleet software assigns missions to robots (backend, machine-to-machine). Open-RMF orchestrates multi-robot task allocation. RRCF defines how operators control and monitor robots (frontend, human-or-AI-to-machine). RRCF sits above these protocols: a RRCF-compliant robot declares its VDA 5050 endpoint within its .rrcf transport block, enabling unified operator control above the existing fleet protocol stack.

3.3 Pillar 3 — Collection & Integration Standard

Because RRCF defines the wire format for all robot commands and telemetry, the same format used for remote control becomes the universal integration format for any external system: data collection pipelines, VLA training datasets, digital twins, simulators, hospital management systems, ERP platforms, safety stacks, and cloud IoT services. One wire format connects robots to the entire software ecosystem — analogous to how USB's common standard enabled any device to connect to any computer.

Because the wire format is symmetric, integration runs in both directions. The same schema that lets a robot stream telemetry out to a collection pipeline lets a pre-recorded or converted command stream be replayed back in — on a declared replay transport endpoint (Section 7.4) — to any RRCF-compliant robot, physical or simulated, exactly as if a live operator or VLA had generated it.

This gives RRCF a general answer to a problem every robotics team otherwise solves separately: getting data collected in one format, on one platform, executed on another. Simulator-generated trajectories, egocentric/human demonstration video, third-party imitation-learning datasets, and any other future data source each need only one converter — source format to RRCF wire format — after which the same replay path plays them back to a real or simulated robot, solving sim-to-real, egocentric-to-real, and imitation-learning-to-real transfer without a separate integration per (source format, robot) pair.

3.4 Pillar 4 — Safety Standard

RRCF mandates E-stop in every .rrcf file. Every compliant robot must halt immediately on estop:true. RRCF also declares watchdog timeouts, geofence boundaries, and speed limits as operational guardrails. RRCF safety declarations complement NVIDIA Halos (IEC 61508

functional safety) and ISO 10218 — Halos ensures the robot does not fail internally; RRCF declares what the robot is permitted to do externally. Together they form a complete safety architecture for Physical AI.

For vehicle-category robots, RRCF also declares speed-gated remote-operator authority tiers: a .rrcf safety block may bound full remote-driving authority to below a declared speed threshold, escalating automatically to remote-assistance-only (monitor plus waypoint or permission grants, no direct actuation) above that threshold. This gives a portable, machine-readable schema for a pattern that UL 4600 flags as a risk category requiring mitigation and that AVSC's Remote Assistance Use-Case best practice describes only as guidance (Section 5, Section 7.3, Section 11).

3.5 Pillar 5 — Choreography Standard

RRCF defines .rrcm (RRCF Mission) files for synchronized multi-robot operations. Pre-computed per-robot trajectories trigger simultaneously via GPS clock sync. Covers drone light shows, robot dance performances, coordinated agricultural swarm operations, and emergency fleet coordination.

4. The Three-Layer Robotics Standard Stack

RRCF completes the three-layer robotics standard stack:

Layer	Standards	What it answers
Physical Layer	URDF, MJCF, SDF, Genesis, USD	What is this robot made of?
Fleet Layer	VDA 5050 v3.0, Open-RMF, AMRA-271	How are missions assigned to robots?
Operator Layer ← RRCF	RRCF — this standard	How does any operator control any robot?

The operator layer was empty before RRCF. Every robot vendor filled it with proprietary solutions. RRCF standardizes it for the first time.

5. Prior Art and Differentiation

Standard	What it covers	Relationship to RRCF	What RRCF adds
VDA 5050 v3.0 (2026)	Fleet mission assignment, JSON/MQTT, wheeled AGV/AMR only	RRCF complements — sits ABOVE in stack	Operator UI declaration, all 12 morphologies, skills, VLA, input modalities
Open-RMF	Multi-robot task allocation, traffic mgmt, ROS2	RRCF complements — different layer	Operator declaration file, non-ROS transports, all morphologies
AMRA-271:2025	Mobile robot comms + peripheral integration	RRCF complements	Operator UI, skill declaration, VLA integration, choreography

URDF / MJCF / SDF	Physical description — kinematics, geometry, physics	RRCF references, does not replace	Behavioral/operator interface layer above physics
NVIDIA Halos (2026)	Functional safety IEC 61508, hardware safety stack	RRCF complements — different layer	External behavioral contract, operator guardrails
UL 4600	Safety case framework for autonomous vehicles; flags teleoperation as a risk category requiring mitigation	RRCF operationalizes	Machine-readable speed-gated authority-tier schema (Section 3.4, Section 7.3) implementing what UL 4600 only describes as guidance
AVSC Remote Assistance	Best-practice guidance for the Remote Assistance use case in automated driving systems	RRCF operationalizes	Declarative remote_assistance vs. remote_driving distinction and control-binding schema (Section 7.3)
USB HID / W3C Gamepad	Raw hardware input — axes and buttons	RRCF provides semantic layer above HID	Robot-specific axis mapping, mode/skill semantics per morphology

6. Morphology Category Taxonomy

RRCF-1.0 defines thirteen canonical robot morphology categories plus an open custom category for future morphologies. A robot declares exactly one primary category. Additional categories may be declared as attachments.

ID	Category	Description	Examples
wheeled	Wheeled / Tracked	Differential, omni, tracked base	TurtleBot, Clearpath Husky, AMRs
legged	Legged (Quad/Hex)	Multi-legged, no arms	Unitree Go2, Spot, ANYmal
loco_manip	Loco-Manipulation	Legged base + arm(s)	Spot+Arm, Go2+Z1, ANYmal+ARM
wheeled_humanoid	Wheeled Humanoid	Wheeled base + full upper body	Sanctuary Phoenix Gen8
full_humanoid	Full Humanoid	Bipedal + full upper body	Unitree G1/H1, Figure 02, Optimus
manipulator	Fixed Manipulator	Stationary arm, no base	UR5/10, Piper AgileX, Franka
aerial	Aerial (Drone/VTOL)	Multirotor, fixed-wing, VTOL	DJI, ArduPilot, PX4 drones
marine_surface	Marine Surface (USV)	Unmanned surface vessel	WAM-V, ArduPilot Boat

marine_sub	Marine Underwater	ROV / AUV subsurface	BlueROV2, VideoRay ROV
industrial_vehicle	Industrial Vehicle	Excavator, dozer, forklift	CAT excavator, Komatsu dozer
agri_vehicle	Agricultural Vehicle	Sprayer, laser weeder, planter	John Deere autonomy, Naïo Oz
road_vehicle	On-Road Vehicle (AV/EV)	Ackermann-steered car/van; steering+throttle+brake control binding; SAE L2–L4 autonomy stack; remote assistance or remote driving	Waymo, Zoox, Tesla FSD, autonomous EV shuttle/robotaxi fleets
custom	Custom (Vendor-defined)	Any future morphology not in above	Medical, nano, space, soft robot

7. RRCF File Structure — .rrcf

A .rrcf file is an XML document. The file may be standalone or embedded within an existing robot description file (URDF, MJCF, etc.):

```
<?xml version="1.0" encoding="UTF-8"?>
<rrcf version="1.0" xmlns="https://rrcf.io/schema/1.0">
  <meta>                                <!-- Robot identity, vendor, physical refs -->
  <primary>                              <!-- Primary category + locomotion declaration -->
  <attachments>                          <!-- 0-N attachments: arm, gripper, sensor, tool -->
  <hud>                                  <!-- HUD/LCD strip + context-sensitive button legend -->
  <skills>                               <!-- Standard + vendor-custom skill declarations -->
  <input_modalities>                    <!-- touch, HID gamepad, voice, gesture, XR -->
  <custom_controls>                     <!-- Up to 5 operator-defined controls -->
  <telemetry>                           <!-- Declared telemetry fields + warn thresholds -->
  <safety>                              <!-- E-stop, watchdog, geofence, speed limits -->
  <transport>                           <!-- MQTT, ROS2, WebSocket, VDA5050, REST -->
  <extensions/>                         <!-- Future RRCF-2.0 blocks, ignored by 1.0 -->
</rrcf>
```

7.1 Complete Example — Unitree Go2 + Z1 Arm

```
<rrcf version="1.0" xmlns="https://rrcf.io/schema/1.0">
  <meta>
    <name>Unitree Go2 + Z1 Arm</name>
    <vendor>Unitree Robotics</vendor>
    <model>Go2-Z1</model>
    <physical_ref format="urdf" path="go2_z1.urdf"/>
    <physical_ref format="mjcf" path="go2_z1.xml"/>
  </meta>
  <primary category="legged">
    <locomotion axes="vx vy wz" max_vx="1.5" max_vy="0.5" max_wz="2.0"/>
    <modes>stand trot bound crawl prone</modes>
    <body_pose pitch="true" roll="true" height="true"/>
  </primary>
```

```

<attachments>
  <attach type="manipulator" id="arm_z1" mount="torso_top" dof="6">
    <modes>stow reach teach replay</modes>
    <ee_control cartesian="true"/>
  </attach>
  <attach type="gripper" id="grip_z1" mount="arm_z1/ee" fingers="2"/>
</attachments>

<hud>
  <field id="mode" position="1" label="MODE"/>
  <field id="battery" position="2" label="BATT"/>
  <field id="estop" position="3" label="STOP" alert="true"/>
  <button_legend>
    <mode name="trot">
      <key id="F1" label="SIT"/> <key id="F2" label="STAND"/>
      <key id="F3" label="BOUND"/>
    </mode>
    <mode name="reach">
      <key id="F1" label="GRIP"/> <key id="F2" label="RELEASE"/>
      <key id="F3" label="STOW"/>
    </mode>
  </button_legend>
</hud>

<skills>
  <skill id="sit" label="Sit" standard="true" cmd='{ "mode": "sit" }'/>
  <skill id="stand" label="Stand" standard="true" cmd='{ "mode": "stand" }'/>
  <!-- Vendor custom - VLA reads at runtime, no retraining needed -->
  <skill id="heel_stretch" label="Yoga Pose"
    description="Extends rear legs in yoga pose"
    cmd='{ "mode": "custom_01" }'/>
</skills>

<input_modalities>
  <modality type="touch_web" enabled="true"/>
  <modality type="hid_gamepad" enabled="true" standard="xinput">
    <axis hid="left_x" rrcf="lx"/>
    <axis hid="left_y" rrcf="ly" invert="true"/>
    <axis hid="right_x" rrcf="rx"/>
    <axis hid="right_y" rrcf="ry" invert="true"/>
    <button hid="A" action="skill:sit"/>
    <button hid="B" action="estop"/>
    <button hid="RB" action="mode_next"/>
  </modality>
  <modality type="voice" enabled="true" language="en-US">
    <phrase text="emergency stop" action="estop"/>
    <phrase text="go home" action="skill:home"/>
  </modality>
  <modality type="gesture_cam" enabled="true" backend="mediapipe">
    <gesture name="fist" action="estop"/>
    <gesture name="thumbs_up" action="skill:stand"/>
  </modality>
  <modality type="xr_controller" enabled="true" api="webxr">
    <hand side="left" maps_to="left_stick"/>
    <hand side="right" maps_to="right_stick"/>
    <trigger side="right" action="skill:grip"/>
  </modality>
</input_modalities>

```

```

<custom_controls max="5">
  <control type="toggle" id="led"          label="LED Strip" default="off"/>
  <control type="slider" id="follow_dist" label="Follow Dist"
    min="0.5" max="3.0" unit="m"/>
  <control type="button" id="bark"          label="Bark" action="momentary"/>
</custom_controls>
<telemetry>
  <field id="battery" unit="%" warn_below="20"/>
  <field id="temp" unit="C" warn_above="55"/>
  <field id="speed" unit="m/s"/>
  <field id="payload" unit="kg"/>
</telemetry>
<safety>
  <estop required="true" topic="/rrcf/go2/estop" qos="2"/>
  <watchdog timeout_ms="500" action="halt"/>
  <speed_limit max_vx="1.5" max_wz="2.0"/>
</safety>
<transport>
  <endpoint role="operator_cmd" protocol="rrcf_mqtt"
    topic="/rrcf/go2/cmd" broker="{MQTT_BROKER}"/>
  <endpoint role="telemetry" protocol="mqtt"
    topic="/rrcf/go2/state" frequency_hz="10"/>
  <endpoint role="ros2" protocol="ros2"
    topic="/cmd_vel" msg="geometry_msgs/Twist"/>
  <endpoint role="fleet" protocol="vda5050" version="3.0"
    topic="/uagv/v2/unitree/go2_001"/>
  <endpoint role="websocket" protocol="websocket" port="9090"/>
</transport>
<extensions/>
</rrcf>

```

7.2 Control-Binding Table

Every RRCF primary category and attachment binds operator input axes to physical actuation through the same pattern: a declared control block maps normalized input axes to actuator-specific targets. Section 7.1's `<locomotion axes="vx vy wz">` (legged base) and `<ee_control cartesian="true">` (manipulator arm) are two instances of this one control-binding table. Section 7.3 adds a third: steering, throttle, and brake for on-road vehicles.

7.3 Vehicle Control-Binding Embodiment — Steering, Throttle, Brake

A `road_vehicle` primary category binds locomotion to steering angle, throttle, and brake instead of holonomic `vx/vy/wz`, and declares whether the active session is `remote_driving` or `remote_assistance`, gated by the safety block's `authority_tier`:

```

<primary category="road_vehicle">
  <locomotion axes="steer throttle brake" max_steer_deg="35"
    max_speed_mps="25"/>
  <modes>remote_driving remote_assistance autonomous</modes>
</primary>
...
<safety>
  <estop required="true" topic="/rrcf/av_001/estop" qos="2"/>
  <watchdog timeout_ms="300" action="halt"/>
  <authority_tier>
    <tier name="remote_driving" max_speed_mps="8" actuation="full"/>
  </authority_tier>
</safety>

```

```

    <tier name="remote_assistance" min_speed_mps="8"
    actuation="waypoint_grant_only"/>
  </authority_tier>
</safety>

```

Below 8 m/s a remote operator may drive directly; above that threshold the interface automatically renders remote-assistance-only controls (route confirmation, waypoint grant, resume) with no direct steering/throttle/brake actuation — the same authority-tier pattern SAE J3016, UL 4600, and AVSC describe in prose, made machine-readable.

7.4 Data Replay — Sim, Egocentric, and Imitation-Learning Data to Any Robot

Because collection and control share one wire format (Section 3.3), a `<transport>` block may declare a replay endpoint alongside its live `operator_cmd` and `telemetry` endpoints. A replay endpoint accepts the same RRCF wire-format command stream — regardless of whether it originated from simulator-generated trajectories, egocentric/human demonstration video, a third-party imitation-learning dataset, or any other source — and plays it back to the declared target, physical or simulated:

```

<endpoint role="replay" protocol="rrcf_mqtt" target="physical|simulation"
  topic="/rrcf/av_001/replay"/>

```

One converter per source format — not one per (source format, robot) pair — is enough to move data from any collection pipeline to any RRCF-compliant robot or simulator.

8. Wire Format

All input modalities — touch, HID gamepad, voice, gesture, XR, VLA — produce the same RRCF wire format output. The robot receives one format regardless of how the operator is interacting:

```

{
  "rrcf": "1.0",          // Protocol version - always present
  "category": "legged",   // Morphology category
  "type": "cmd",          // Message type
  "lx": 0.75,             // Left stick X range [-1.0, 1.0]
  "ly": 0.00,             // Left stick Y range [-1.0, 1.0]
  "rx": -0.25,            // Right stick X range [-1.0, 1.0]
  "ry": 0.00,             // Right stick Y range [-1.0, 1.0]
  "speed": 0.60,          // Speed scalar range [0.0, 1.0]
  "mode": "trot",         // Active locomotion mode
  "skill": null,          // Active skill id or null
  "estop": false,         // E-stop - true = halt immediately
  "input": "touch_web",   // Which modality produced this command
  "twist": {              // ROS2 geometry_msgs/Twist - always present
    "linear": {"x":0.75, "y":0.00, "z":0.00},
    "angular": {"x":0.00, "y":0.00, "z":-0.25}
  },
  "ts": 1719000000000     // Unix timestamp milliseconds
}

```

8.1 Ecosystem Interoperability (Informative)

RRCF does not model its wire format as a subset of any single dataset framework's SDK. Interoperability is handled at the boundary instead of by inheritance: a LeRobot [LEROBOT] robot configuration MAY be imported as one device's .rrcf declaration, and RRCF-collected episodes MAY be exported in LeRobot's own dataset format for compatibility with its ecosystem and model hub. Implementers exporting per-camera video SHOULD follow LeRobot's own chunked, index-based naming convention rather than embedding task text in filenames; task text belongs in a separate per-episode manifest, not the media path.

For live visualization and debugging, an RRCA implementation MAY expose the open Foxglove WebSocket protocol [FOXGLOVE] directly — a transport-only, schema-agnostic protocol with no built-in authentication. An implementation that exposes it MUST secure it at the gateway boundary using the same bearer-token mechanism as other RRCF transport endpoints (Section 13), rather than a vendor-specific agent stack.

9. VLA Integration — RAG for Robots

RRCF enables Vision-Language-Action models to control any compliant robot via Retrieval-Augmented Generation. VLA integration has two structurally different tiers, and an implementation MUST NOT represent one as the other.

9.1 Tier 1 — Discrete Skill Invocation (Zero-Shot)

- At session start, the VLA loads the robot's .rrcf file as structured context.
- Matching a natural-language instruction to a declared skill_id and emitting its pre-authored cmd is standard tool-calling and requires no training on RRCF specifically.
- Vendor-custom skills (heel_stretch, seed_row_align) become available to the VLA instantly because the skill declaration IS the context — no retraining.
- This solves the combinatorial explosion for discrete skills: thousands of robots x hundreds of skills = millions of skill invocations impossible to pre-train, made available at runtime instead.

9.2 Tier 2 — Continuous, Parametric Control

Producing a pose delta, a joint target, or any other continuous value not covered by a pre-authored skill cmd requires the VLA's output head to already be conditioned on RRCF's declared action space — normalized appropriately, in the declared units, respecting declared joint and actuator limits. The .rrcf file supplies the target action space; it does not supply the unnormalization statistics a continuous-control policy needs to produce values in that space. Those statistics live in the policy's own training checkpoint, not in the robot's declaration. An implementation MUST NOT claim zero-shot continuous control from .rrcf context loading alone: retrieval-augmented loading gives zero-shot access to the declared skill vocabulary (Tier 1); it gives a trained policy a well-defined target for Tier 2, not a substitute for that training.

9.3 Conformance Tiers

- RRCF-compliant — any off-the-shelf VLA paired with an external adapter that unnormalizes, remaps, and clamps its native output into RRCF's declared action space, using the policy's own checkpoint statistics. Works with any model; translation logic stays outside the standard.
- RRCF-native — a VLA fine-tuned so its output head's target space is RRCF's declared units directly. The adapter reduces to validation, clamping, and actuator transform only.

A higher conformance bar and a stronger interoperability claim; maps to a distinct Foundation certification tier (Section 12).

9.4 Scope of Benefit (Informative)

The table below states plainly where a declared, shared action space helps and where it does not, to avoid overclaiming.

Benefit	RRCF's contribution
Better raw VLA perception	No inherent benefit
Better reasoning	No inherent benefit
Lower model inference latency	No; can be worse if an extra translation step is introduced
Fewer action-translation errors	Yes
Consistent units and semantics	Yes
Explicit joint/actuator constraints	Yes
Safety validation	Yes
Robot/vendor integration	Strong
Cross-robot dataset compatibility	Strong
Cross-embodiment policy reuse	Potentially very strong
New robot onboarding	Potentially very strong
Foundation-model training across robots	Potentially very strong

RAG for Robots: instead of retrieving documents, the VLA retrieves the robot's declared skill vocabulary — zero-shot. Producing continuous control in that same space is a property of how the policy was trained, not of loading the file.

10. Input Modalities

RRCF-1.0 defines five required/optional input modality types, plus three haptic and neural modalities planned or in draft for RRCF-1.1+, and an open extension point for any future input format. All produce identical wire format:

Modality	Technology	RRCF-1.0	Notes
touch_web	Browser touch / mouse	Required — baseline	On-screen joysticks rendered from .rrcf
hid_gamepad	W3C Gamepad API / USB HID	Optional	Xbox/PS5/any HID. Mapping declared in .rrcf
voice	Web Speech API	Optional	Skill names become voice commands automatically
gesture_cam	MediaPipe / camera	Optional	Hand gestures mapped to RRCF actions

xr_controller	WebXR Device API	Optional	XR hand pose maps to joystick axes + skills
tactile_glove	Force-feedback haptic glove (multi-channel tactile sensing)	Planned — RRCF-1.1	Per-finger force/contact mapped to gripper/end-effector control bindings
tactile_arm	Force-feedback haptic arm / exoskeleton controller	Planned — RRCF-1.1	Joint-torque and pose feedback mapped to manipulator control bindings
bci	Brain-computer interface (EEG/EMG or implant-derived intent signals)	Draft — RRCF-1.1+	Discrete decoded intent mapped to skill/mode actions; full neural_interface morphology support targeted for RRCF-3.0 (Section 12)
custom	Any future input format not yet standardized (e.g. eye-tracking, EMG wristband)	Reserved extension point	Declared via <modality type="custom"> with vendor-defined schema; compliant controllers MUST NOT reject unrecognized modality declarations (Section 11.2)

11. Conformance Requirements

11.1 RRCF-Compliant Robot MUST:

- Include a valid <rrcf version="1.0"> block in robot description or standalone .rrcf file.
- Accept RRCF wire format JSON on at least one declared transport endpoint.
- Implement E-stop: upon estop:true, halt ALL motion immediately, no exceptions.
- Implement watchdog: halt if no valid RRCF command within declared timeout.
- Publish telemetry on declared fields at minimum 1 Hz.
- For road_vehicle category, enforce the declared authority_tier: reject direct steering/throttle/brake actuation above the tier's speed threshold, falling back to remote-assistance-only commands.
- Accept RRCF wire format commands on a declared replay endpoint identically to live operator_cmd input, subject to the same E-stop and watchdog enforcement.

11.2 RRCF-Compliant Controller MUST:

- Parse .rrcf file without requiring any other description format.
- Render core panel (joysticks, E-stop, speed, telemetry) for ALL categories.
- Render profile panel from declared category, modes, skills, attachments.
- Render HUD strip from <hud> fields and update button legend per active mode.
- Support touch_web input modality as baseline.
- Include estop field in EVERY transmitted command message.

- Normalize all input axes to range [-1.0, 1.0] in wire format output.
- Include rrcf version string and Twist sub-object in every command message.
- Parse and ignore, rather than reject, <modality> declarations of unrecognized type (forward compatibility for future input formats).
- Render a remote-assistance advisory panel (no direct actuation controls) when the active authority_tier is remote_assistance.

11.3 Scope Principle

Every field RRCF defines, including the extensions in Sections 14–16, is admitted by one recurring test: does this fact affect how a command is interpreted, or whether it is safe to execute? If so, it belongs in RRCF and MUST be enforced by the same real-time watchdog as every other safety-relevant field. If a fact is inventory, scheduling, or business data — a maintenance ticket, a warranty term, a purchase order — it belongs at the platform layer above RRCF, referenced by a stable identifier (unit_id, ref) rather than duplicated inside the declaration.

12. Versioning and Governance

RRCF uses semantic versioning. RRCF-1.0 is the initial standard. Certification authority is held by Indo-Mars Technologies / MarsGeo Platform. Category proposals follow: proposal → 60-day review → draft → 3 independent implementations → ratification → publication.

Version	Planned additions	New categories
RRCF-1.0	13 categories, all 5 pillars, 5 input modalities, vehicle control-binding, speed-gated authority tiers, data replay endpoint, environment declaration, base/overlay/device composition pipeline, identity/lifecycle metadata, two-tier VLA conformance	wheeled, legged, loco_manip, wheeled_humanoid, full_humanoid, manipulator, aerial, marine_surface, marine_sub, industrial_vehicle, agri_vehicle, road_vehicle, custom
RRCF-1.1	Voice modality refinements, gesture spec, XR training, tactile_glove/tactile_arm haptic input, bci (draft)	No new base categories — attachment type additions
RRCF-2.0	World context block, benchmarking block, RL reward hints	medical_endoscopic, surgical, micro_robot, exoskeleton, soft_robot, swarm_node
RRCF-3.0	Neural interface, zero-G thruster primitive	nano_robot, space_zero_g, neural_interface

13. Security Considerations

- Transport encryption: MQTT MUST use TLS (MQTTTS port 8883). WebSocket MUST use WSS in production.

- Authentication: Endpoints SHOULD require auth. Wire format contains no auth fields — handled at transport layer.
- E-stop hardware independence: Robots MUST implement hardware watchdog independent of network.
- VLA guardrails: Controllers and VLA integrations MUST reject commands exceeding declared safety limits.
- Command rate limiting: Robots SHOULD implement rate limiting against denial-of-service.

14. Environment Declaration

A robot rarely operates alone. Much of what determines whether a command is safe — whether GPS is available, whether the workspace is indoor or outdoor, where the geofence boundary sits — is a property of the space a robot is in, not the robot itself. RRCF-1.0 defines an environment block for this: inlined for a standalone robot, or referenced for a fleet operating in a shared space.

```
environment: { space_type: indoor, localization: {...}, ... } # standalone case

environment:
  ref: warehouse-A # fleet case, same
schema
```

14.1 Environment Fields

- `space_type` — indoor / outdoor / mixed; structured / unstructured.
- `localization` — declared per capability (gps, rtk, slam, gsm), each with source: inbuilt | attachment | environment_provided, plus a reference id when environment-provided.
- `degradation_behavior` — the action the safety layer MUST take if a declared localization source becomes unavailable, enforced by the same E-stop/watchdog machinery as Section 3.4 and Section 7.3.
- `bounds / geofence` — the operating envelope for any robot referencing this environment.

14.2 Declared State vs. Runtime State

An environment declaration has a declared half and a state half. The declared half — which localization sources exist, where boundaries sit, how the space is laid out — composes with a robot's own declaration through the mechanism in Section 15. The state half — whether a fix currently holds, which zone the robot is currently in — is a live companion read alongside the resolved declaration and MUST NOT be merged into it at compose time.

14.3 Ownership of Cameras, Sensors, and Compute

One test decides whether a camera, sensor, or compute resource is declared in a robot's own .rrcf file or in the Environment artifact: is its pose fixed relative to the robot frame, or relative to the room? A wrist or gripper camera moves with the robot and MUST be declared in that robot's own file, under the per-camera table below. A ceiling camera, a front-of-workspace camera, or any other room-fixed sensor or compute resource MUST be declared once in the Environment artifact and referenced by every robot operating in that space rather than duplicated per robot. The same test applies to compute: an edge-inference box mounted on an arm is declared in that robot's own file, in a section distinct from the control/telemetry-binding tables of Section 7; a shared GPU rack serving a warehouse is declared once, in the Environment artifact.

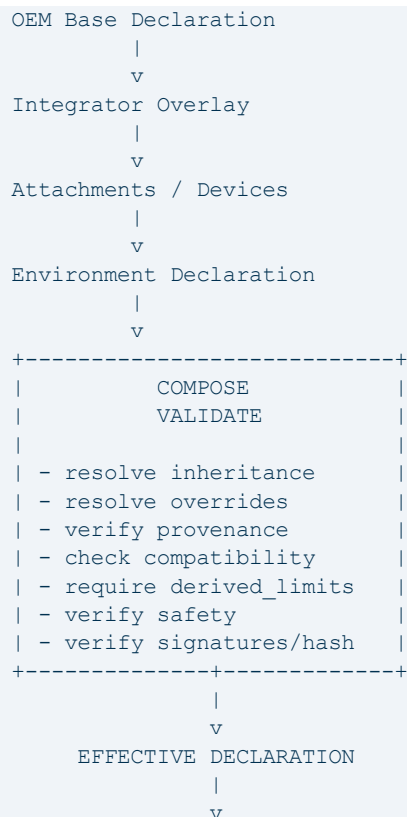
Field	Description	Example
id	Semantic key	wrist, top, front_left
purpose	Manipulation-proximal vs. exteroceptive/scene	grasp_view, scene_overview
resolution / fps / color_mode	Capture parameters	1280x720, 30fps, rgb
mount_point	Attachment point, or “environment-fixed”	arm_z1/ee, environment-fixed
intrinsics/extrinsics ref	Calibration reference, if calibrated	calib/wrist_cam_001.yaml

14.4 Sensor Declarations

The same pattern — a semantic id, a unit, a sample rate, a mount point — applies to non-visual sensors such as force/torque and tactile arrays, following the named-key, per-sensor configuration convention already common in robot-learning tooling.

15. Declaration Composition

A .rrcf file is not always authored by one party. An OEM publishes a base declaration for an arm; an integrator adds a third-party gripper and a customer-specific safety envelope; the assembly then operates inside a specific warehouse. RRCF-1.0 resolves this through a five-stage pipeline: Declare, Extend, Compose, Validate, Execute.



```

RRCA
|
v
Robot

```

Two additional data flows sit alongside the Effective Declaration but are never merged into it at compose time: Runtime State (the robot's own live status) and Observations (its raw sensor streams). Section 14.2 applies identically here — both are read in parallel with the resolved declaration, not folded into it.

Runtime State	Observations
+-- joint state	+-- camera
+-- battery	+-- LiDAR
+-- localization	+-- IMU
+-- RTK fix	+-- force
+-- watchdog	+-- etc.
+-- etc.	

15.1 Base-Plus-Overlay Composition

A downstream document **MUST NOT** edit or fork an upstream declaration. It references the base by {ref, version, hash} and declares only the delta, following the same pattern as Kustomize, Helm, and Linux Device Tree overlays:

```

base: { ref: oem/acme-arm-x1, version: 2.3.0, hash: sha256:... }

devices:
- id: third_party_gripper
  mass: 1.2kg
  reach_delta: 150mm
  affects_payload: true

derived_limits:
  applies_to: [third_party_gripper]
  certification: integrator_declared # or oem_certified
  safe_payload: 3.8kg
  safe_reach: 950mm

environment:
  ref: warehouse-A

```

15.2 Mandatory Validation

Any devices entry that affects payload or reach (affects_payload: true, or equivalent) **MUST** carry a corresponding derived_limits block in the resolved declaration. A compose/validate implementation **MUST** refuse full-capability operation for a resolved declaration missing a required derived_limits block, applying the same enforcement posture as the mandatory E-stop and watchdog fields of Section 3.4.

15.3 Provenance

A derived_limits block **MUST** carry a certification field: oem_certified for vendor-published, signed compatibility data, or integrator_declared for a self-computed figure. Both are enforced identically at runtime; the distinction is visible to the certification authority (Section 12) and **MUST NOT** be hidden or normalized away during compose.

15.4 Scale Invariance and Service Events

This mechanism is scale-invariant: a simple deployment MAY collapse everything into a single `robot.rrcf`; a complex deployment MAY split into `oem.rrcf`, `customer.rrcf`, `environment.rrcf`, and `devices.rrcf` files. Both resolve through the same schema and algorithm — file count is a deployment choice, not a schema difference. A service event that changes a safety-relevant fact (a swapped actuator, a recalibration, a different end effector) MUST produce a new pinned version and hash and an updated `derived_limits` block; service tooling SHOULD emit this automatically rather than relying on manual reinstallation of a prior file.

16. Identity and Lifecycle

Composition (Section 15) resolves a declaration for a class of hardware; identity resolves it for one physical unit.

16.1 Unit Identity

A `unit_id` (serial number) anchors the base: `{ref, version, hash}` resolution of Section 15 to a specific physical robot, and MUST be distinct from `model` or `type`, which describe a design rather than a physical object. Manufacture date is informational metadata and MUST NOT drive runtime logic.

16.2 Lifecycle Fields

A small, safety-scoped set of lifecycle fields — `last_verified`, `service_interval` / `service_due` — MAY be declared alongside identity. When declared, a compliant runtime MUST use them to degrade capability or warn when a unit is operating past its declared verification window, and MUST feed them back into the composition pipeline of Section 15 rather than treating them only as a historical log.

16.3 Scope Boundary

Full maintenance-ticket detail — what was replaced, by whom, warranty terms, technician notes — is explicitly out of scope for RRCF. This is fleet-management data, referenced by `unit_id`, and belongs at the platform layer above RRCF (Section 11.3), not inside a file a real-time watchdog has to parse.

16.4 Alignment with Product-Passport Regulation (Informative, Open)

Robots are likely to fall under product-passport regulation of the kind already adopted for other product categories in the European Union [DPP] regardless of RRCF's own choices. Aligning Identity and Lifecycle field names to that emerging standard, rather than inventing bespoke ones, is an open design question targeted for resolution before RRCF-1.1 (Section 12).

17. References

- [RFC2119] Key words for RFCs, Bradner, 1997.
- [VDA5050] VDA 5050 v3.0, VDA/VDMA, March 2026.
- [OPEN-RMF] Open Robotics Middleware Framework, OSRA, 2024.
- [AMRA-271] AMRA-271:2025, Autonomous Mobile Robot Alliance, 2025.
- [ANSI-A3] ANSI/A3 R15.06-2025, A3/ANSI, October 2025.
- [HALOS] NVIDIA Halos Robotics Safety System, NVIDIA, June 2026.
- [W3C-PAD] Gamepad API, W3C, 2024.

- [WEBXR] WebXR Device API, W3C, 2024.
- [SAE-J3016] Taxonomy and Definitions for Terms Related to Driving Automation Systems, SAE International, 2021 (rev. 2026).
- [UL4600] UL 4600, Standard for Safety for the Evaluation of Autonomous Products, UL Standards & Engagement, 2023.
- [AVSC-RA] AVSC00006202008, Considerations for Remote Assistance in Automated Driving System (ADS) Operations, SAE AVSC, 2020.
- [DPP] Regulation (EU) 2024/1781 (Ecodesign for Sustainable Products Regulation), European Parliament and Council, 2024; Digital Product Passport harmonized standards, Commission Implementing Decision (EU) 2026/1736.
- [LEROBOT] LeRobot, Hugging Face, 2025.
- [FOXGLOVE] Foxglove WebSocket Protocol, Foxglove Technologies, 2024.
- [RRCF-REF] RRCF Reference Implementation, Indo-Mars Technologies, July 8, 2026.

Author

Moorthi PV | Indo-Mars Technologies / MarsGeo Platform
info@indo-mars.com | indo-mars.com | rrcf.io (pending)

Copyright (c) 2026 Moorthi PV. Apache License 2.0.